Basic-Level Interview Q&A: C#, ASP.NET MVC, Web API, Azure PaaS

1. C# Fundamentals

Q1. What is the difference between value types and reference types? Value types (`int`, `double`, `bool`, `struct`) store the actual data directly and are stored on the stack. Reference types (`class`, `string`, `array`, `object`) store a reference (pointer) to the data, which lives on the heap. Assigning a value type copies the data; assigning a reference type copies the reference.

Q2. Abstract class vs Interface — when to use which? An abstract class can have both implemented and unimplemented members, fields, and constructors, but a class can inherit only one. An interface only declares members (though default implementations are allowed since C# 8) and a class can implement multiple interfaces. Use an abstract class when classes share common base behavior/state; use an interface when you just need to guarantee a contract across unrelated classes.

Q3. What are async/await, and why avoid `.Result`/`.Wait()`? `async`/`await` let you write non-blocking asynchronous code that frees up the calling thread while waiting on I/O (DB calls, API calls, file access). Calling `.Result` or `.Wait()` on a Task blocks the calling thread synchronously, which can cause deadlocks (especially in ASP.NET) and defeats the purpose of async.

Q4. Difference between IEnumerable, ICollection, and IList?

- `IEnumerable`: read-only forward iteration only (no Count, Add, Remove).
- `ICollection`: extends IEnumerable, adds Count, Add, Remove, Contains.
- `IList`: extends ICollection, adds index-based access (`list[i]`), Insert, RemoveAt.

Q5. What is `IDisposable` and the `using` statement for? `IDisposable` is implemented by classes that hold unmanaged resources (file handles, DB connections, streams) so they can be released deterministically via `Dispose()`. The `using` statement automatically calls `Dispose()` when the block ends, even if an exception occurs.

Q6. Explain boxing and unboxing. Boxing converts a value type into a reference type by wrapping it in an `object` (stored on heap). Unboxing extracts the value type back out. Both have a performance cost and should be avoided in hot paths — generics (`List<int>` instead of `ArrayList`) help avoid boxing.

Q7. What are nullable types and why use them? `int?` (shorthand for `Nullable<int>`) allows a value type to also hold `null`. Useful for representing "unknown" or "not set" data, especially when mapping to database columns that allow NULL.

Q8. Difference between `const` and `readonly`? `const` is a compile-time constant, must be assigned at declaration, and is implicitly static. `readonly` is assigned at runtime (in the constructor or at declaration) and can differ per instance.

Q9. What is LINQ? Give a simple example. LINQ (Language Integrated Query) lets you query collections using SQL-like syntax directly in C#.

```csharp
var activeEmployees = employees
    .Where(e => e.IsActive)
    .OrderBy(e => e.Name)
    .ToList();
```

Q10. How does exception handling work in C#?

```csharp
```

```
try
{
    // risky code
}
catch (SqlException ex)
{
    // handle specific exception
}
catch (Exception ex)
{
    // handle general exception
}
finally
{
    // always runs (cleanup code)
}
```

You can also define custom exceptions by inheriting from `Exception`.

---

2. ASP.NET MVC

Q1. Explain the MVC pattern.

- **Model**: represents data and business logic.
- **View**: the UI (Razor `.cshtml` files) that renders the model.
- **Controller**: handles incoming requests, interacts with the model, and selects a view to return. This separation makes code more testable and maintainable.

Q2. What is routing, and attribute routing vs convention-based routing? Routing maps incoming URLs to controller actions. Convention-based routing is defined centrally (e.g., `{controller}/{action}/{id}` in `RouteConfig`). Attribute routing places the route directly on the action:

```csharp
[Route("employees/{id}")]
public IActionResult Details(int id) { ... }
```

Q3. Difference between ViewData, ViewBag, and TempData?

- `ViewData`: dictionary, passes data from controller to view for a single request, requires casting.
- `ViewBag`: dynamic wrapper over ViewData, same lifetime, no casting needed.
- `TempData`: persists data across one redirect (survives one subsequent request), stored in session by default — useful for messages after a redirect (e.g., "Saved successfully").

Q4. What are Action Filters? Attributes that run logic before/after an action executes. Examples: `[Authorize]` (authentication check), `[ValidateAntiForgeryToken]`, or a custom filter for logging:

```csharp
public class LogActionFilter : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        // log before action runs
    }
}
```

Q5. Basic Razor syntax to loop through a model list.

```html
```

```
@model List<Employee>
<ul>
@foreach (var emp in Model)
{
    <li>@emp.Name - @emp.Department</li>
}
</ul>
```

Q6. What is Model Binding and Model Validation? Model Binding automatically maps incoming form/query/route data to action method parameters or a model object. Model Validation uses Data Annotations to enforce rules:

csharp

```
public class Employee
{
    [Required]
    public string Name { get; set; }

    [Range(18, 65)]
    public int Age { get; set; }
}
```

Then in the controller: `if (!ModelState.IsValid) { return View(model); }`

Q7. Difference between ActionResult, ViewResult, and JsonResult? `ActionResult` is the base type an action can return. `ViewResult` returns an HTML view. `JsonResult` returns JSON data. Other derived types include `RedirectResult`, `ContentResult`, `FileResult`.

Q8. What is a Partial View and when do you use it? A reusable chunk of a view (e.g., a header, a comment list, a card component) rendered with `@Html.Partial("_EmployeeCard", employee)` or `<partial name="_EmployeeCard" model="employee" />`. Useful to avoid duplicating markup across views.

Q9. What is Dependency Injection in .NET, and how is it configured? DI is a design pattern where dependencies (services) are provided to a class rather than created inside it, improving testability and loose coupling. In `Program.cs`/`Startup.cs`:

```csharp
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
```

Then inject via constructor:

```csharp
public class EmployeeController : Controller
{
    private readonly IEmployeeService _service;
    public EmployeeController(IEmployeeService service) => _service = service;
}
```

Q10. MVC vs Web API project types? In classic ASP.NET Framework, MVC and Web API were separate frameworks with separate base classes (`Controller` vs `ApiController`). In ASP.NET Core, they're unified — a single `Controller` base class handles both, and you differentiate by whether you return Views or data (JSON).

---

3. Web APIs

Q1. What is REST, and what are the standard HTTP verbs? REST (Representational State Transfer) is an architectural style for designing stateless APIs over HTTP.

- `GET` — retrieve data
- `POST` — create new data
- `PUT` — replace/update an entire resource
- `PATCH` — partially update a resource

- `DELETE` — remove a resource

Q2. Difference between PUT and PATCH? `PUT` replaces the entire resource with the payload sent (missing fields may be nulled out). `PATCH` applies a partial update, changing only the specified fields.

Q3. How do you return proper HTTP status codes?

csharp

```csharp
[HttpGet("{id}")]
public IActionResult Get(int id)
{
    var emp = _service.GetById(id);
    if (emp == null) return NotFound();        // 404
    return Ok(emp);                             // 200
}

[HttpPost]
public IActionResult Create(Employee emp)
{
    _service.Add(emp);
    return CreatedAtAction(nameof(Get), new { id = emp.Id }, emp); // 201
}
```

Q4. What do `[ApiController]` and `[Route]` do? `[ApiController]` enables API-specific behaviors: automatic 400 responses on invalid model state, attribute routing requirement, and inference of parameter binding sources. `[Route]` defines the URL pattern for the controller/action:

csharp

```
[ApiController]
[Route("api/[controller]")]
public class EmployeesController : ControllerBase { ... }
```

Q5. How do you handle exceptions globally in a Web API? Using middleware:

```csharp
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsync("An error occurred.");
    });
});
```

Or a global exception filter implementing `IExceptionFilter`.

Q6. What is Swagger/OpenAPI? Swagger (OpenAPI) auto-generates interactive API documentation and a test UI from your controllers/models. In .NET, enabled via `AddSwaggerGen()` and `UseSwaggerUI()`. Useful for developers to explore and test endpoints without Postman.

Q7. How do you secure a Web API? Common approaches:

- **JWT (JSON Web Token)**: client authenticates once, receives a signed token, sends it in the `Authorization: Bearer <token>` header on each request.
- **API Keys**: simple shared secret in headers, often for service-to-service calls.
- **OAuth2**: delegated authorization, commonly used with Azure AD/Entra ID for enterprise apps.

Q8. What is CORS and why configure it? CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a web page from calling an API hosted on a different domain unless the API explicitly allows it. Configure in `Program.cs`:

```csharp
builder.Services.AddCors(options =>
    options.AddPolicy("AllowFrontend", policy =>
        policy.WithOrigins("https://myapp.com").AllowAnyMethod().AllowAnyHeader()));
```

Q9. How would you version an API? Common strategies: URL versioning (`/api/v1/employees`), query string (`?api-version=1.0`), or header-based versioning. .NET has the `Microsoft.AspNetCore.Mvc.Versioning` package to manage this cleanly.

Q10. Why use `async Task<IActionResult>` for endpoints? It keeps the request thread free while waiting on I/O-bound operations (database calls, external API calls), improving scalability under load — especially important for APIs handling many concurrent requests.

---

4. Azure PaaS

Q1. IaaS vs PaaS vs SaaS — where does App Service fit?

- **IaaS**: you manage VMs, OS, patching (e.g., Azure Virtual Machines).
- **PaaS**: platform manages OS/runtime; you manage just the app and data (e.g., **Azure App Service**, Azure SQL Database).
- **SaaS**: fully managed software you just use (e.g., Microsoft 365). Azure App Service is PaaS — you deploy code, and Azure handles the underlying servers, OS patching, and runtime.

Q2. What is Azure App Service, and how do you deploy a .NET app to it? A fully managed service for hosting web apps and APIs. Deployment options: Visual Studio "Publish" wizard, Azure DevOps/GitHub Actions CI/CD pipeline, ZIP deploy, or `az webapp deploy` via Azure CLI.

Q3. What is an App Service Plan, and scale up vs scale out? The App Service Plan defines the underlying compute resources (VM size, region, pricing tier) that host one or more App Services. **Scale up** means moving to a more powerful tier/VM size (more CPU/RAM). **Scale out** means adding more instances running in parallel behind a load balancer (horizontal scaling).

Q4. How do you manage configuration/secrets in Azure?

- **App Settings**: environment variables in the App Service configuration blade, override `appsettings.json` values per environment.
- **Connection Strings**: dedicated section for DB connection strings, can be encrypted.
- **Azure Key Vault**: centralized, secure storage for secrets/certificates; the app retrieves them at runtime via Managed Identity rather than storing secrets in config files.

Q5. What is Azure SQL Database vs self-hosted SQL Server? Azure SQL Database is a fully managed PaaS relational database — Microsoft handles patching, backups, high availability, and scaling. You don't manage the OS or SQL Server instance directly, unlike a self-hosted SQL Server on a VM.

Q6. What is Application Insights used for? An Azure monitoring service (part of Azure Monitor) that tracks application performance, exceptions, request rates, dependency calls, and custom telemetry. Integrated via a NuGet package/SDK, it helps diagnose issues in production without attaching a debugger.

Q7. What is a deployment slot, and why is it useful? A deployment slot is a separate live instance of your App Service (e.g., "staging") with its own hostname. You deploy and test new code in staging, then use **slot swapping** to move it to production with zero downtime and an easy rollback path if something goes wrong.

Q8. Azure Functions vs App Service? Azure Functions is a **serverless**, event-driven compute service — you write individual functions that run in response to triggers (HTTP request, timer, queue message) and pay only for execution time. App Service hosts a full always-on application (or is billed per the App Service Plan) and is better suited for full web apps/APIs with consistent traffic.

Q9. How do you set up CI/CD to deploy to Azure? Using Azure DevOps Pipelines or GitHub Actions: on every commit/PR to `main`, a pipeline builds the .NET project (`dotnet build`), runs tests (`dotnet test`), publishes artifacts (`dotnet publish`), and deploys to the App Service (via `azure/webapps-deploy` action or an Azure DevOps release pipeline), often deploying first to a staging slot before swapping to production.

Q10. What is Azure Blob Storage, and when would you use it? A PaaS object storage service for unstructured data — files, images, documents, backups. In a .NET app, you'd use it (via the `Azure.Storage.Blobs` SDK) to store uploaded files (e.g.,

employee profile photos, generated reports) instead of storing them in the database or local disk, which doesn't scale well across multiple App Service instances.

Practical Walkthrough Question (commonly asked)

"Walk me through what happens from a browser request to the database and back in an MVC app."

1. Browser sends an HTTP request to a URL.
2. **Routing** matches the URL to a Controller and Action.
3. **Model Binding** maps request data (form/query/route values) to action parameters.
4. **Model Validation** runs (Data Annotations) — if invalid, return the view with errors.
5. The Controller calls a **Service/Repository layer**, which uses **Entity Framework** to query/update the **database**.
6. Data returns to the Controller as a Model.
7. Controller passes the Model to a **View** (Razor) or returns JSON (for an API).
8. The View renders HTML using the Model data, which is sent back to the browser.

Tip: If you want, I can turn this into a timed mock interview (I ask one at a time, you answer, I give feedback) so you can practice actively instead of just reading.